

HFS

2411:INTRODUCT
ION TODATA
SCIENCE LECTURE
2

J KAROMO

Expected Learning Outcomes

- At the end of the Module, Learners should be able to;
 1. Outline the various Computer Programming Languages that are Out there.
 2. Explain Why Python is the Language of Choice for Data Scientists.
 3. Describe the Standard Python Indentations and Standard Python Variables
 4. Describe the use of Loops (For & While) for repetitive Tasks such as in Automating Tasks.
 5. Discuss the Application of Loops, Control Statements (IF) and Conditional Statements to Make Decision.
 6. Discuss the combination of Block of Codes, Loops, Control Statements, Conditional Statements, in the building of Automation Scripts and/or Softwares.

Review of Week One

- The role of a Data Scientist is to turn raw data into actionable Insights, build Models and Algorithms and to crunch Big Data to make sense of it.
- To be an effective Data Scientist, One must know how to extract, process, structure, manipulate, analyze data using Python and other Languages.

INTRODUCTION TO COMPUTER ROGRAMMING AND PROGRAMMING LANGUAGES 1

1. **Computer Software:-** Collection of instructions that enable a user to interact with a computer or perform a specific task (e.g. of Softwares include; System, Programming & Applications)
2. **Computer Application:-** A Program, or a set of codes or Computer language designed or structured for performing specific and highly Specialized Work, Tasks, Processes, Procedures by a Computer such as Spreadsheets, Word Processing, Databases, etc.
3. **A Code:-** Refers to “*A Set of Instructions written using an Appropriate Language to Computer for performing a Specific Task,*” OR Alternatively “*A System of Rules to Convert one Form of Information into Another Form /Representation/Output by a Computer*”.
4. **Coding/Programming:-** Writing a set of instructions/rules/codes for performing specific tasks/output/information representation by a computer.

INTRODUCTION TO COMPUTER PROGRAMMING AND PROGRAMMING LANGUAGES 2

5. Types of Codes:-

1. **Source Code:-** Refers to Primary/Initial/Original Code that is the Source of A Programme, before Version Management, in Collaborative/Group Software Development Projects.
2. **Source Coding:-** Source of a program, that originally tells the Programme/Computer how to function
3. **Pre-compiled code:-** A precompiled or inbuilt code that is borrowed/imported for use
4. **Scripts:-** List of commands consisting of set of Codes defining and specifying Variables, their Values and Arguments for Performing Specialized Tasks, Activities, Actions, Processes, Procedures by a Computer, that are executed by a certain Application or Program (e.g. Automate or generate web pages)
6. **A Syntax:-** Structure/arrangement of text/words/numbers/statements to create a code
7. **Programming Language:-** Is a Platform with sets of Syntax for Writing Codes/Rules/Instructions for a computer, with accessible Inbuilt Functions, Modules and Libraries for making the Writing Easier. A Computer Programming Language is an Enabler of Data Science, Artificial Intelligence and Big Data Analytics as they run on Codes, Functions, Models and Algorithms

INTRODUCTION TO COMPUTER PROGRAMMING AND PROGRAMMING LANGUAGES 3

- Some of the Programming Languages out there include;
 1. Python
 2. R/Ruby,
 3. Rusty
 4. Java,
 5. Java Script,
 6. C
 7. C++
 8. HTML,
 9. PHP,
 10. Clojure
 11. HTML, etc.

WHY LEARN CODING WITH PYTHON FOR DATA SCIENCE 1

- However, despite the numerous programming languages out there, Python is the Language of Choice for Data Scientists, due to the following Reasons;
 1. Simple Syntax, that Prioritizes Code Readability.
 2. Several Automation Scripts, for automating Repetitive and Boring Tasks/Processes or Procedures.
 3. Specialized Libraries such as for Data Extraction, accessible to Users, when they get stuck.

WHY LEARN CODING WITH PYTHON FOR DATA SCIENCE 2

4. Several accessible Libraries, Inbuilt Functions, Modules live templates for repetitive tasks.
9. Integrated Frameworks for Deep Learning .
10. Integrated Frameworks for web and Mobile Phone (Android System based) applications development.
11. OOP or Code-based on need or any Problem as Objects with Properties and Behaviors.
12. Simple Syntax for beginners
13. Inferences, decision making
14. Flexible analysis, Can analyze Data Sets by class distribution.

IMPORTANT FEATURES OF PYTHON 1

1. Open-Source Nature, Free Download from Python Homepage.
2. General Purpose OOP with Broad Variety of Applications (Instagram, WhatsApp, YouTube, interest, Quora Platforms).
3. Many Applications for Web Development, Mobile (Android) Applications, Gaming, Software Development, Machine Learning, Data Science, Platforms and Back-end Development.
4. High Capacity for Scaling Complicated Systems (Scalability)
5. Cross-validation of data in model training and testing thus quality assurance for reliability of model.
6. Ability to evaluate performance metrics of models upfront

IMPORTANT FEATURES OF PYTHON 2

7. Ability to optimize models performance accuracy
8. Ability to evaluate several models performance accuracy concurrently.
9. Several inbuilt Functions and Modules which are reusable, thus shortening your code and “**DRY**” Concept.
10. Use of pre-defined or Inbuilt Functions that are easily accessible to Users.
11. Ability to Create user-defined Functions and save them as Static Files for Future Reference that are accessible by use of “Import Statements” from directory/Library.
12. A clearly defined Syntax that is Simple, hence quick to learn.

IMPORTANT FEATURES OF PYTHON 3

13. Capability to Interface with major Databases-API Integration and Other Languages.
14. Unlike the Others, the use of Indentation (Single or Double Quotes) to define a Basic Code, Set or a Block of Codes (e.g. List or String or a Script).
15. Space Indentation to Define/Indicate the Block of Code, a Statement Belongs to.
16. Frameworks for Developing Basic Web Pages (Flask) and Mobile Phone Applications (Django).
17. Other Frameworks for Creating “Neural Networks” in Deep Learning (PyTorch & TensorFlow).

GETTING STARTED WITH PYTHON

- Getting Started entails:

1. Downloading of the Free Pycharm (Community version) from the Python homepage and Interpreter, instructions for installation and licence activation.
2. Navigating the User interface and Projects in Pycharm by creating a new project, giving it a title, choosing interpreter and specifying its location directory.
3. Pycharm thinks in terms of Projects and sub-projects and all projects are configured to use interpreter.



Getting Started with PyCharm: Quick Tour

Rectangular Snip



GETTING STARTED WITH PYTHON

1. Hence can create new project or open an existing one.
2. A “User Interface” or “Window Editor” opens by default when a new project is created which allows the user to write, edit, run, navigate, synchronize, refactor a Code r order a return of finished task (in Flask).
3. A special window below the Window Editor called “Python Console” is where codes are executed after running the code, configurations are managed, shows modules and route path of your project and any error in the code.

```
1 from flask import Flask
2 # creating a flask app
3 app = Flask(__name__)
4
5 @app.route('/IT')
6 def IT():
7     x = 6
8     y = 4
9     answer = x+y
10    print(answer)
11    return 'Wellcome to IT'
12
13 @app.route('/Business')
14 def Business():
15     x = 5
16     y = 4
17     answer = x*y
18    print(answer)
19    return 'Karibu Business'
20
21 @app.route('/Engineering')
    Business()
```

Full-screen Snip

Run: Lesson 1 Functions

"C:\Users\Mburu Samuel\PycharmProjects\Mburhtmlflask\venv\Scripts\python.exe" "C:/Users/Mburu Samuel/PycharmProjects/Mburhtmlflask/Lesson 1 Functions.py"
* Serving Flask app "Lesson 1 Functions" (lazy loading)

WARNING: This is a development server. Do not use it in a production deployment.

Use a production WSGI server instead.

* Debug mode: off

* Running on <http://127.0.0.1:5000/> (Press CTRL+C to quit)

Run: 6: TODO Terminal Python Console

Event Log

IMPORTANT FEATURES OF THE USER INTERFACE/WINDOW EDITOR

1. **Structure:** On the top left of the window which gives the structure of Projects and sub-projects, break points and book marks.
2. **Favourites:** On the bottom left of the window that gives the frequently opened projects or sub projects.
3. **Step-up Operations:** at the bottom of the Python console that has numerous tabs for organizing tabs, customizing tools e.g. Version Controls.

IMPORTANT FEATURES OF WINDOW EDITOR

4. Databases: on the top right of the window where you can add, change interpreters, synchronize or customize tools.
5. Line Numbering: Used for intentional stopping or pausing a programme for Debugging purpose.

NAVIGATING THE WINDOW EDITOR

TASK BAR 1

1. Pycharm Preferences: where you can customize theme or coding intentions
2. File: where you open or save new projects changes to existing projects. Templates/Functions/Codes created are saved in “Static file”.
3. Edit: for making changes to existing projects
4. View: for different formats of viewing projects
5. Navigate: for quick access to existing projects and growing through a code.

NAVIGATING WINDOW EDITOR TASK BAR 2

6. Refactor: for returning running codes or renaming projects
7. Run tab: for Implementing or Executing a Code
8. Tools: contains numerous tools for organizing or customizing projects
9. VCS or the Version Controls: with step-up operations tools for enhancing the coding productivity.

NAVIGATING WINDOW EDITOR TASK BAR 3

- 10. **Window Tab:** with tools for navigating the Window
- 11. **Help:** for requesting for assistance with the application

PYCHARM TOOLS & RESOURCES

- Has many built-in Functions, Modules, live Templates, Libraries stored in python library/directory that can be easily accessed when stuck or for easing repetitive tasks e.g. quick documentation, random mailing/messaging etc.
- Running and navigating a code is by right clicking and clicking “run” the python file, whereas files can be quickly accessed by using their path, name, defined class or symbols.

NAVIGATING THE PYTHON CONSOLE

1. Where codes, debugging are executed and errors in the code and Route path of your project is shown.
2. Step up operations for productive coding
3. Version Control for organizing Tabs .
4. Debugging a programme for tracking error that helps developers monitor and fix errors/crashes in real time e.g. pdb.

CODING WITH PYTHON:- GETTING STARTED 1

1. Python is case-sensitive except for key words e.g. True, False None etc.:- mostly uses lower case.
2. Python does not accept spaces:- Combine texts/words or use underscore or commas to separate items e.g. “hello_there” or hellothere.

STANDARD PYTHON VARIABLES: REVIEW OF THE FUNDAMENTALS

- A “Variable” in Python refers to *“A Reserved Memory Space/Location to Store Parameter/Variable Values, that can be accessed any time”*.
- Hence Many types of Variables such as Numbers, Strings, Lists e.g. of Names, Numbers, Answers, Content, Sum, Numbers, Expressions, Loos, Dictionaries, Functions, Modules, Classes, Objects, Instances, User, Applications, etc.
- The first step in writing a basic Code, is to;
 1. **Declare/Create the Variable or Variables that you are going to work with, e.g. a number variable and assign it a Value.**
- For Example, $X = 10$, where X is the Variable and 10 is the Value stored in that Variable that is accessible any time
- Or Name = John, where name is the Variable and John, the Value stored in that Variable.
- Or a List of Names e.g. Names = (John, Samuel, Joy, Jane ,David).
- **Every Bracket in Python Must be Enclosed.**

Declaring/Creating and Storing Variables

- To Declare/Create and Store a Variable e.g. a String in Python, you assign Values to them e.g. `String = Hallo Forensic Science Students`, or a Class Register for Forensic Science Year 4 Students, e.g. `Class_Register = ForensicsY4`.
- The Logic behind Declaring a Variable and assigning a Value to it, is to enable Python to not only store the Variable, but also to remember and access that Variable later, when called upon.
- When you Assign a Value to a Variable in Python, you reserve some Memory Space/Location, so that, from that instant, that Variable is accessible any time it is called upon. Hence, can store numerous types of Variables.
- The equal sign (`=`), is used to assign “Value” to Variables, e.g. `Marks = 45`
- The Operand on left of the equal sign is the **Name of the Variable**, while that on the right is the **Value stored in the Variable**.

FUNDAMENTALS OF CODING

1. Concept of Defining Variables, Assigning Values for Reserving Memory Locations and Specifying Arguments for a Code/Script.
2. Object-Oriented Approach to Programming, the DRY Concept, Creating Classes.
3. The Standard Python Variables, Operators and Operands, For and While Loops, Control Statements.
4. Decision Making with “IF Statements” or Conditional Statements.
5. Importing In-built Functions, Modules and Libraries using the “Import Statement”, or Building User-Built Functions/Module.
6. Automations Scripts, Python Libraries, Functions and Modules.

KEYWORDS AND IDENTIFIERS 1

- A “Keyword” is a reserved word in Python, hence cannot be used as Keyword for Variable or Function Name or as any other Identifier.
- Keywords are Case sensitive, thus all Keywords except TRUE, FALSE and NONE are always in lower cases.
- Some of the Keywords in Python include; **class, return, continue, for, while, from, del, and, not, with, if, elif, or yield, assert, else, import, pass, break, except, in, raise, yield.**

KEYWORDS AND IDENTIFIERS 2

- An “Identifier” is a name given to entities like a class, function, variables to differentiate one entity from the other.
- It can be a combination of letters in lower cases or an underscore e.g. `forensicScience`, `forensic_Science`, `var_1`, however cannot start with a digit/number.
- Camel-Case style of writing is acceptable, that is capitalizing every first letter of a group of words except the initial word, but without spaces e.g. `forensicScienceYear4`.
- Unlike other Programming languages that uses Braces(`{}`), to Define a Basic Code, Set or Block of Codes, Python uses Indentation (Single or Double Quotes).

STANDARD PYTHON INDENTATION 1

- Standard python indentation include:-
 1. Double Quotes (“ ”):- Used to define a Block of Codes, text (“Hallo”), word(s)(“I am a string”), Strings (“Hallow_there”), Textual Variable in a List [“Kiambu”, “Nyeri”, “Kisumu”, “Nairobi”], Variables in Dictionary {“Name”: “Zara”, “Age”:7}, Variable being printed print(“You_passed”), and to Enclose Input Variable such as a=int(input(“Date of Birth”))

STANDARD PYTHON INDENTATION 2

2. **Colon [:]** :- Used to separate “Keywords” in a Dictionary and to end a user-defined function/code e.g. `def_add (a+b):`
3. **Square Bracket []**:- Used to enclose all items in a list
4. **Normal Bracket ()**:- Used to enclose Variables to be printed, in a Function e.g. in 2 and Input Variables e.g. in 1.
5. **Braces { }**:- Used to enclose all Items in a Dictionary.
6. **Comma (,)**:- Used to separate Items in a Dictionary (e.g. in 1) or list (e.g. in 1).
7. **Space Indentation:-** Mandatory spacing requirement for defining/indicating what block of code or to which block of code a “Statement” belongs.
8. **#:-** is used to make a comment.

VARIABLES:- PYTHON STANDARD VARIABLES

- In Python, a “**Variable**” refers to “*Something whose Value or Memory Location can Change, or a Container that can Store Something that can be Accessed Anytime*”.
- Python works with five standard variables:-
 1. **Numbers**:- These are numerical values, which can be either integers (whole numbers) or floats (point real value or with decimal points).
 2. To work with the numbers, you must first define whether the number is an integer or float.

STANDARD PYTHON VARIABLES

- 5 Standard Python Variables;

1. Numbers

1. Integers
2. Floats

2. Strings

3. Lists & Loops

1. For Loops
2. While Loops

4. Tuple

5. Dictionary

NUMBER VARIABLES

- A “Numerical Variable” is a single, two, or more than two numerical values, and when 2 or more, separated by Commas enclosed by Square Bracket ([]).
- Two main types of numbers stored in Python;
 1. Integers are whole numbers
 2. Float are numbers with decimal places
 3. Others e.g. Complex/Compound Numbers
- A Number Variables is created by declaring/defining the Variable such as giving it a name “X” and Assigning a Value to the Variable/Name “X”, e.g. X=10.
- A “Number Variable” can be a single Number, or a List of Numbers e.g. Numbers = [1,2,3,4,5,6,7,8,9,0].
- For User Specified Inputting a Numerical Variable, the Number Type (Whether Integer or Float) must be specified.
- X= Integer(Input(25)) or Y= Float(Input(23.5)).

VARIABLES:- NUMBERS 2

- While Integers are whole numbers, floats have decimal places which must be defined always, as under;
- `a=int(input (variable))` or
- `b=float(input(variable))`
- Note the 2 closing brackets (every bracket should be closed).
- Notably, Numbers not indented, but when more than one, they are separated with Commas.

VARIABLES:- NUMBERS 3

- To print numbers:
- `print(10)`, you do not have to specify the type of number to print.
- To specify the user input such as adding or removing numbers, you can use the input function e.g;
- `a=int(input(enter value of a number e.g.10))`
- The input function is used to add number 10 into the variable.

VARIABLES:-NUMBERS:- OPERANDS, OPERATORS AND OUTPUT

- Operators are special symbols in Python used for arithmetic, comparison/relational or logical computation.
- Examples Include:- $+$, $-$, $=$, $*$, $\%$, $<$, $>$, $\geq$ etc.
- For instance, In the equation $2 + 3 = 5$, the “2” and “3” are the Operands, “+” and “=” are the Operators and “5” is the Output.

OPERATORS

- Operators are special symbols in Python for carrying out specific operations e.g. Arithmetic, Logical Computations, etc., that operates on Operands.
- They are used with Operands in Arithmetic, with Strings, If Statements to Make Decisions or Conditional Statements.
- There are Several operators in Python for various operations, including;
 1. Basic Arithmetic Operators, such as +, -, *,/, etc.
 2. Comparison Operators, that compares Values or Variables, such as >, <, ==, %=, !=, <=, >= . A Double Equal Sign is used to compare variables (Not equal to). %= means Divisible by 2, != means Not divisible by 2, etc.
 3. Assignment Operators, for Assigning Values to Variables, such as = e.g. a=5,
 4. Logical Operators, such as AND, OR, NOT etc.
 5. Membership Operators, to test if a value or variable is found in a sequence, such as in (5 in X) out, except, including, etc.
- Operators can either be defined or accessed from the Libraries.

VARIABLES:- ADDING NUMBERS USING THE INPUT FUNCTION

- E.g. Numbers= [20,30,50.5]
- 1. `a=int(input(20))`, i.e. input/add an integer from the user
- 2. `b = int(input (30))`
- 3. `C = float(input(50))`
 - Print the numbers:- `print(a+b+c)`
 - Adding the two numbers:-
 - Can print directly, e.g. `print("sum is", a+b+c)` or can specify first the answer, e.g. `answer = 20+ 30+ 50.5` and then print the answer e.g.
 - `print("sum is", answer)`

VARIABLES:- NUMBERS:- SPECIFYING USERS ACTION

- Can specify what the user is supposed to do by adding a string inside the input bracket e.g.
`a=int(input("Enter value of first number"))`
- `b=int(input("Enter value of second number"))`
- Or `a=int(input("Date of Birth"))`
- `Print("sum is", a+b)`
- `Print("Date of Birth")`

STRINGS VARIABLES

- A “String” is a contiguous sets of characters (Texts, Words, Phrases, Statements, or a mixture of Words and Numbers, etc.) e.g. Names or Greetings indented with double quotes and separated by commas.
- A String Variable is created by Declaring/Defining/Naming the String e.g. giving it a Name e.g. “a” and Assigning the String Name a Value that will be Stored in a Reserved Memory Space/Location and will be accessible Later, when called upon.
- For instance in a =“Hallothere” or “Hallo_there”, “a” is the String Variable/Name, while “halloThere” or “Hallo_there”, are the stored words (or the Value of the String Variable).
- Notably, a “String Variable” is not enclosed in Brackets, but Only enclosed in Double or Single Quotes,
- Can display, print, add or delete text in a String, by use of display, print, addedum and del or Remove Functions.

VARIABLES:-ADDING OR DELETING ITEMS FROM STRINGS

- It is also Possible to add and delete items from a String.
- Subsequently, the following code is used for adding items to a string:`marklist.append(item)`.
- Where “marklist” is the name of the string and “item” is the item you want to add or delete from the String.
- Likewise, the following code can be used for deleting an item from a string:`marklist.remove(item)`.

LISTS AND LOOPS VARIABLES

- A “List” contains items (Texts, Names, Strings, Words, Values, Numbers, etc.) separated by Commas and all enclosed within Square Brackets [].
- To create a “List Variable”, The List Must first be declared/defined/named and assigned Values, for Storage.
- Lists are Dynamic (Ever Changing), thus you can add or remove items to the List using the append or del or remove functions.

LOOPS- INTRODUCTION

- A Loop is an array that can store many strings or numbers in a list.
- An infinite or never ending array/sequence or list or iterating over elements of lists, tuple and dictionary.
- Process of repeating a task n-times or when you want the computer to repeat a certain task or similar task multiple times.
- Three Types:-
- **While Loop:-**
 1. This is a Control Flow Statement that allows a code to be executed repeatedly based on a given condition (as long as the condition is met)
 2. Alternatively, it is a repeating “IF Statement”.
 3. Repeating a task until the condition becomes false or is no longer met.
 4. The “While Loop” checks the condition before the code is executed, if the condition is true, the Code repeats until the condition becomes false
 5. i.e. a While Loop consists of a block of code and a condition.

LISTS AND LOOPS VARIABLES

- A Loop is a continuous or never ending (Infinite), list of items or sequence.
- Statements in Python are executed sequentially, in an Orderly Manner, that is first, second, third, etc.
- To execute a block of codes several number of times, control structures/Statements to allow for the more complex execution, are needed.
- Loops Statements allow the execution of the Statements or Group of Statements several times.
- Consequently, we have the “While Loop”, “For Loop” and the “Nested Loop”, Statements.

LISTS AND LOOPS VARIABLES

- while loop repeats a Statement continuously, if a given condition is TRUE, by testing the condition first and then executing the Loop body.
- for Loop executes a sequence of statements several times, abbreviates the Code that manages the Loop Variable.
- A Nested Loop is a Loop Domiciled inside a While or a For Loop.
- Hence, A Nested Loop uses one or more Loops inside either a while or for Loops.

TUPLE VARIABLES

- A Tuple contains a number of fixed items (names, values etc.), separated by Commas, and all the items enclosed within Braces/Parentheses {}.
- Unlike the String, List and Dictionary that are Dynamic, you Cannot add or remove an item from a Tuple.

DICTIONARY VARIABLE

- A Dictionary Stores details of Items/Objects using a “Key” and the “Item Value”.
- The Key is double/single quoted, separated from its Value by a Colon (:) and the Double or Single Quoted Values.
- A Comma is used to separate the details of one Key from the other Key and all the items in the Dictionary enclosed within curly Braces { }.
- To create a Dictionary, first declare/give it a name and then assign Values to the Items of the Dictionary, for Storage.
- Can add or remove items in a Dictionary using the add, del or remove functions.
- Can also access items/Values from a Dictionary, can use the .get Method, e.g. `print(items.get("a", none))`

VARIABLES:- DICTIONARY 2

- In a Dictionary, the Keyword is indented with a single(' ') or double quote (" "), separated by a Colon (:), while the items or particular details of the keyword are separated by comma.
- And the whole dictionary is enclosed in curly braces { }.

VARIABLES:-DICTIONARY 3

- To work with dictionary in Python, you must define the dictionary for Python, as thus;
- `dict = {'Name': 'Samuel', 'Age': 45, 'Gender': 'Male', 'Contact': 0789234563},`
- Where “dict” is the variable and “Name”, “Age”, “Gender”, “Contact” are the values assigned memory space, or stored in the memory in Python.

VARIABLES:- DICTIONARY 4

- You can then print the whole Dictionary or specific values of the same.
- To print the whole dictionary, use;
- `print(dict)`
- The following code can be used for printing Dictionary values;-
- `print("Your name",dict["Name"])`

VARIABLES:- DICTIONARY 5

- Likewise, you can add and delete items to a Dictionary.
- The following codes are used to add and delete an item to a dictionary, respectively, as thus:
- `append.dict["Item"]`
- `del.dic["Item"]` e.g. "Occupation" or "Employment Status"]

BASIC ARITHMETIC 1

- To perform a basic arithmetic, declare all the Number Variables and Assign them Values for storage.
- Can use the Combination or Addition function to declare the answer/output.
- For instance, to perform addition of the numbers 10, 20, 30, declare the variables and assign the values, e.g.
- `num1=10`
- `num2=20`
- `num3=30`
- Alternatively
- `a=10`
- `b= 20`
- `c= 30`

BASIC ARITHMETIC FUNCTION

2

- Can define/declare the answer, e.g.
- `answer=num1+num2+num3` in Combination Method or can print directly in the addition Method e.g.
- `print(answer)` OR
- `print(a+b+c)`

LOOPS CODES:- WHILE LOOPS 1

EXAMPLES

1. The While Loop Statement is “whileexpression: statement(s)”
2. Used to repeat Strings of a number of times as long as the Condition is TRUE.
3. To initialize a While Loop, you first declare and assign values to the Loop and then specify the starting loop e.g.

```
loop=[0,1,2,3,4,5,6,7,8,9,10]
```

```
X=0
```

- Then can set the condition (control) to stop the Loop or not e.g. to end at 9 or 10, such as

```
while X < 10 or X =10, Then print the Statement e.g.
```

```
print(“Good_Morning ForensicsY4”, X).
```

- To stop the Loop you set Control or Condition by making the Condition of the Loop FALSE e.g. adding or subtracting 1 e.g. X=X+1.

EXAMPLES OF LOOPS

1. While($l > 5$): where $l = \text{integer}$
 - Print (i) :- out put 0,1,2,3,4
2. While ($i \geq 6$):
 - Print i :- output 0,1,2,3,4,5,6,
1. Printing all odd numbers between 0 and 100
 - Counter = 0 where counter is the lower limit
 - While(counter = ! 2 = > 100):
 - Print (counter) :- output all odd numbers between 0 and 100

WHILE LOOPS 2

3. Writing a while loop to print from 1900 to 2019
 - While(counter = ! 1900 = 2019):
 - Counter = 1900
 - Print (counter) :- out put = all years between 1900 to 2019

FOR LOOPS 1

for Loop:-

1. Repeating a task for multiples times
2. Are used to iterate/repeat over elements of a sequence (in a List, Tuple or Dictionary).
3. Used when a block of code is to be repeated a number of times.
4. Distinguished by an explicit loop counter/loop variable.
5. To initialize a for Loop, First Declare the Loop, give it a name e.g. Marklist and assign Values to the Variables for Storage, then can print all the items or specific ones in the Loop a number of times, or can add an "If Statement" to the for Loop.

EXAMPLES OF LOOPS

- e.g. Marklist= [23, 14,23,24,17,21,18,19,25]
- to print all Marks in the list;
- for X in Marklist:
- print X
- OR adding an “IF Statement” to print only those marks greater than 20;
- if X>20:
- print(X)output=X

FOR LOOPS 2

3. For loop to print from -6 to 6) or in range (-6, 6):
 - For ii(-6,6): out put -6,-5,-4,-3.....0, 1,2....6)
4. To print element in a list
 - E.g. Items = ["apple", 1,4, "exit", "321"]
 - For item in items:
 - Print (item) output = apple 1,4 exit 321

FOR LOOPS 3

4. Use of zip function to Traverse/combine 2 or more sequences
 - E.g. names = ["Alice", "Bob", "Trudy"]
 - Hobbies = ["painting", "Singing", "Hacking"]
 - Ages = [21, 17, 22]
 - Combine the sequences using zip () function as follow;
 - For persons, age, hobby in zip (names, ages, hobbies):
 - Print (person + "is" + str (age) +" years old and his/her hobby is" + hobby)

FOR LOOPS 4

- Output =
- Alice is 21 years old and his/her hobby is painting
- Bob is 17 years old and his/her hobby is singing
- Trudy is 22 years old and his/her hobby is hacking

FOR LOOPS 5

5. To repeat an if statement in a list of marks
 - E.g. marklist = [45, 78, 45, 45, 12, 22, 89, 55, 22, 74, 45, 65]
 - For x in marklist:
 - Print X
 - If $x < 50$:
 - Print (“less than 50”)
 - Else print (“great than 50”)
 - Output --45 less than 50, 12 less than 50, 78 greater than 50 etc.

FOR LOOPS 6 EXAMPLES

- To write a For Loop:- for i(lower limit, upper limit): loop body where `lii` = integer, `%` = divisible by 2, `!` = not divisible by 2.
- 1. E.g. For counter range (1,6):
 - Print “counter”)
 - Or numbers = range (1,6)
 - For count in numbers:
 - Print(“count) output 1,2,3,4,5
- 2. For texts or words
 - Word = “computer”
 - For letter in word
 - Print(“letter”) output computer

DECISION MAKING

- It is the anticipating of a condition occurring while executing the programme and specifying of actions according to the condition
- Evaluate expressions which produce True or False outcome

DECISION MAKING:- IF AND IF....ELSE STATEMENTS 1

- If... statement:- used when the condition is true e.g. Marks=45 return answer “Passed” or “D” when the condition is true
- If..... else, are optional statements when the expression is false e.g. If marks < 50 return answer “Failed” or “E” When the condition is false

DECISION MAKING:- IF AND IF....ELSE STATEMENTS 2

- Arriving at a failed decision if the marks < 50 , hence, when the condition is true such as if a marks = 45 is encountered an output of failed is returned e.g.
- Print("You Failed")
- If else checks if the condition is true

DECISION MAKING:- ELSE OR ELIF STATEMENTS 3

- ELSE or ELIF statement applies when there are multiple conditions to be met e.g. ≥ 50 to ≤ 60 return answer “Credit” or “B”
- Else:
- Print(“Credit”) when the condition is true

Useful Websites and Platforms

1. FreeCodeCamp
2. ChatGPT
3. Blackbox.AI
4. Synthesia.io
5. Copy.AI
6. Google Search Engine
7. YouTube.



THANK YOU